

FIT3143 Lab 1: Generative AI Prompt Records Log

Authors: Erwyna Soo Wen Xin (36555789) & Taabish Farooq Bhat (35473932)

Unit: FIT3143 Parallel Computing Lab 1 Assessment

AI Tool Used: Google Gemini

Task 1: Serial Code Optimisation & Correctness

Prompt 1.1 (Algorithmic Logic): "How can I implement an optimized trial division algorithm in C to find all prime numbers strictly less than n that skips even numbers, checks odd divisors up to square root of k , and writes to an array to maintain ascending order without sorting?"

Prompt 1.2 (Verification & Timing): "How do I verify that my C code correctly identifies exactly 664,579 prime numbers for $n = 10,000,000$ and accurately measures execution time in seconds using `clock_gettime()`?"

Task 2: POSIX Threads Parallelisation & Tuning

Prompt 2.1 (Pthreads Setup & Lock-Free Execution): "Write a C code structure using POSIX Threads (`pthread_create` and `pthread_join`) that distributes prime checking across p threads using a lock-free boolean flag array to avoid mutex overhead."

Prompt 2.2 (Workload Partitioning Analysis): "Why does cyclic chunking (e.g., chunks of 1000 in rotation) outperform single large block partitioning when distributing prime checking work across POSIX threads?"

Task 3: OpenMP Implementation & Workload Scheduling

Prompt 3.1 (Pragma Directives & Compilation): "How do I parallelise a prime-checking loop in C using OpenMP directives like `#pragma omp parallel for schedule(dynamic, 1000)`? What compiler flag is needed (`-fopenmp`), and how does OpenMP handle load balancing?"

Prompt 3.2 (Scheduling Schemes): "What are the performance trade-offs between OpenMP static, dynamic, and guided scheduling schemes when computing non-uniform tasks like trial division up to $n = 30,000,000$?"

Task 4: Benchmarking, Data Plotting, and Presentation Defense

Prompt 4.1 (Benchmarking Automation): "Help me write a Bash script `run_benchmarks.sh` to execute Serial, Pthreads, and OpenMP binaries across 30 values of n (from 1M to 30M) and thread counts 1 to 28, averaging 3 runs per datapoint."

Prompt 4.2 (Visualization Scripting): "Write a Python script `make_graphs.py` using matplotlib to plot all 8 required graphs comparing runtime and speedup against problem size n and thread counts p , including ideal linear reference lines."

Prompt 4.3 (Rubric Audit & Hardware Analysis): "Check our slide deck against the FIT3143 Lab 1 rubric to ensure we meet HD criteria for graph completeness, hardware saturation analysis (14 cores vs 28 threads), and presentation clarity."